

ARM for x86 People – FYP Prep Guide

Updated after actual setup on Kay's machine (July 17, 2026).

Toolchain corrected: prebuilt ARM GNU toolchain in `/opt/arm-toolchain` with prefix `arm-none-linux-gnueabi-`, not the AUR `arm-linux-gnueabi-` package (which failed to build/clone). All commands below use the working prefix. Exercise 1 is confirmed done and correctly diagnosed (prologue/epilogue, fp vs r7, Thumb code density, blx/mode-switch via the Thumb bit on `_IO_puts`, and the encoding-form-dependent S flag on Thumb-1 MOV variants).

Target: read Ghidra disassembly of Qualcomm firehose agent binaries and trace what they're doing. Firehose agents are almost always Thumb-2 (Cortex-A/Cortex-M territory), sometimes with ARM-mode fallback code, so you need both.

0. The one mental shift that matters more than any instruction

x86 is a register-memory architecture: `add [rbx], eax` touches memory directly. ARM is **load/store**: the ONLY instructions that touch memory are `LDR*` / `STR*` (and `LDM` / `STM` / `PUSH` / `POP`). Every arithmetic/logic instruction operates on registers only. This is why ARM code looks "chattier" than x86 – you'll see a `LDR` to pull a value into a register, do the op, then `STR` it back. Once this clicks, ARM disassembly stops looking alien.

Second shift: there is no dedicated flags-setting behavior by default. x86 `add` / `sub` / etc. always set flags. ARM instructions only set flags (N,Z,C,V in CPSR) if you explicitly suffix them with `S` (`ADDS`, `SUBS`, `ANDS`...) – plain `ADD` does not touch flags. This matters when you're trying to figure out what a `CMP` downstream is actually testing.

Third: almost every ARM instruction can be **conditionally executed** (encoded in the instruction itself in ARM mode, or via **IT** blocks in Thumb-2). x86 only conditionalizes branches (**jz**, **jne**...). ARM conditionalizes *anything*: **ADDEQ**, **MOVNE**, **STRGT**. Ghidra will show these as separate mnemonics with suffixes – don't be confused, they're not new instructions, they're **<opcode><condition>**.

1. ARM vs Thumb-2 – when each is used

- **ARM (A32)**: fixed 4-byte instructions, all 32 bits used for encoding (fewer instructions fit in cache, but simpler to decode). Full condition-code-on-every-instruction, full barrel shifter on operand2.
- **Thumb / Thumb-2 (T32)**: mix of 2-byte and 4-byte instructions. Thumb-1 (ARMv4T/v5) was pure 16-bit, limited register access (mostly r0-r7), no conditional execution except branches. **Thumb-2** (ARMv6T2+, what you'll actually see on modern Cortex-A/Snapdragon EDL/PBL/firehose code) mixes 16- and 32-bit encodings, has near feature-parity with ARM mode, and adds the **IT** (if-then) instruction to conditionalize up to 4 following instructions.
- **Why it matters for you**: Qualcomm bootloader/firehose code targets code density (small SRAM/IMEM footprint at boot), so it's compiled Thumb-2 almost universally. Cortex-A53/A7 application processors used in Snapdragon EDL/Sahara boot stages run Thumb-2 for PBL/SBL and often firehose too. You WILL also hit ARM-mode stubs (interrupt vectors, some crypto/hardware init) so you need to read both.
- **Interworking**: switching between ARM and Thumb state happens via **BX** / **BLX** where **bit 0 of the target address** signals mode (1 = Thumb, 0 = ARM). This is why you'll see odd addresses like **0x08001234 | 1** in vector tables and function pointers in Ghidra – that low bit isn't part of the real address, it's a mode flag. Ghidra usually handles this automatically and shows you disassembled Thumb correctly, but if you ever see garbage disassembly, suspect a mode-detection failure and manually retype the function as Thumb (Ghidra: right-click → "Disassemble as Thumb").

2. Register file – mapped to x86

ARM has 16 general-purpose-visible registers, r0-r15, 32-bit each (AArch32 – firehose/EDL is AArch32, not AArch64, since it targets pre-XBL boot code and older cores; confirm per-target but assume 32-bit unless told otherwise).

ARM	AAPCS name	x86-64 rough equivalent	Notes
r0	a1	rax (return) / rdi (1st arg)	1st arg AND return value register
r1	a2	rsi (2nd arg)	2nd arg / high word of 64-bit return
r2	a3	rdx (3rd arg)	3rd arg
r3	a4	rcx (4th arg)	4th arg, also often used as scratch
r4-r8	v1-v5	rbx, r12-r15 (callee-saved)	callee-saved, general purpose
r9	v6/SB/TR	–	callee-saved OR platform register (varies by ABI variant – see below)
r10	v7/SL	–	callee-saved
r11	v8/FP	rbp	frame pointer , but only by convention, not enforced like rbp historically was
r12	IP	–	no x86 equivalent – intra-procedure-call scratch register, used by linker veneers/interworking stubs, NOT preserved across calls
r13	SP	rsp	stack pointer, always full-descending, always word-aligned
r14	LR	(return address is on x86 stack, not a register)	link register – holds return address, set implicitly by <code>BL</code> / <code>BLX</code> . Big divergence from x86: no implicit push happens
r15	PC	rip	directly visible and writable as a GPR – you can <code>MOV pc, r0</code> to jump. Reads as current instruction + 4

ARM	AAPCS name	x86-64 rough equivalent	Notes
			(ARM) or +4 (Thumb, pipeline effect) – matters for PC-relative literal pool loads

There's also **CPSR** (Current Program Status Register) – combines what x86 splits across `rflags` (condition flags N/Z/C/V) plus mode bits (including the Thumb bit, bit 5).

The LR difference is the single biggest gotcha coming from x86. On x86, `call` pushes the return address onto the stack automatically; `ret` pops it. On ARM, `BL` just copies `PC+4` into `LR` – nothing touches the stack. This means **leaf functions** (that don't call anything else) often never touch the stack at all, they just `BL` ... no wait, they don't even need to save LR since they don't call out – they just `BX LR` to return directly from the register. Non-leaf functions must manually save LR to the stack on entry (typically via `PUSH {..., LR}`) and restore+return in one shot via `POP {..., PC}` – popping directly into PC instead of doing a separate return instruction. When you see `POP {r4-r7, pc}` in Ghidra, mentally read that as `leave; ret` combined.

3. AAPCS calling convention (the ARM equivalent of cdecl/stdcall)

This is what actually lets you trace function calls in a stripped Ghidra binary.

Argument passing:

- `r0`, `r1`, `r2`, `r3` hold the first 4 integer/pointer arguments.
- 5th and further arguments go on the stack, pushed by the *caller*, in order (first extra arg at lowest address – same direction convention as cdecl).
- 64-bit args (long long, double without VFP hardfloat) consume a register **pair**, and must start on an even register (`r0:r1` or `r2:r3`) – if a 64-bit arg would start on `r1` or `r3`, it's skipped and the value goes in the next even-aligned slot/stack. This is a classic

source of "why does this function skip r1" confusion in Ghidra – check for this alignment rule before assuming a hidden argument.

- Structs passed by value: if small enough, packed into registers; otherwise passed by reference to a caller-allocated copy. Ghidra's decompiler usually flags this reasonably well but verify against the actual LDR/STR pattern if it looks off.

Return values:

- r0 for values ≤ 32 bits.
- r0:r1 for 64-bit values.
- Larger aggregates (structs) returned via a hidden pointer passed in r0 by the *caller* (caller allocates space, passes pointer as an invisible first arg, real args shift to r1+). If you see a function whose first visible "argument" in Ghidra's decompilation looks like it's writing to a struct pointer and doesn't match the source signature you'd expect – this is why.

Register preservation (the actual "calling convention" table you'll use constantly):

Register	Preserved across call?	Ghidra will show it as
r0-r3	No (caller-saved / argument regs)	scratch, args
r4-r8, r10, r11	Yes (callee-saved)	if a function pushes these on entry and pops on exit, it's using them as locals across a call
r9	Depends on platform variant (RWPI/TLS use) – assume callee-saved unless proven otherwise on your target	
r12 (IP)	No	scratch, veneer/PLT stub usage
r13 (SP)	Yes, by definition	
r14 (LR)	No (caller must not assume it survives a call – but see below)	

Stack:

- Full-descending, grows downward, SP always points at last valid item (not one-past like some ABIs).
- **8-byte aligned at any public function boundary** (a stricter requirement than x86's 16-byte-at-call-time rule but same idea) – this is why you'll see `SUB sp, sp, #8` type padding even for small local variable allocations.
- Function prologue pattern to recognize instantly in Ghidra:

```
PUSH {r4, r5, r7, lr}      ; save callee-saved regs you'll clobber +
return addr
...body using r4,r5 as locals...
POP {r4, r5, r7, pc}       ; restore + return in one instruction
```

This is your `push rbp; mov rbp, rsp; ... ; pop rbp; ret` equivalent – except which registers get pushed varies per function based on what it actually uses, since ARM doesn't mandate a frame pointer the way old x86 code did. r11 (FP) *may* be set up as a frame pointer (`ADD r11, sp, #N` after the push) if compiled with frame pointers enabled (`-fno-omit-frame-pointer`) – Qualcomm boot code is frequently built with frame pointers omitted for size, so don't expect r11 to reliably be a frame pointer; treat SP-relative offsets as ground truth instead.

- Syscalls / supervisor calls happen via `SVC #imm` (was `SWI` in old ARM mode) – equivalent conceptually to x86 `syscall` / `int 0x80`, though on bare-metal firehose/EDL code you won't see OS syscalls, you'll see SVC used for secure-monitor calls into TrustZone or similar, or plain function calls to boot ROM/SBL service routines.

4. Key instructions you'll actually hit in Ghidra

Data movement:

- `MOV Rd, Rm` / `MOV Rd, #imm` – like `mov`. Note: ARM (pre-Thumb2 32-bit imm support) historically couldn't encode arbitrary 32-bit immediates in one instruction (12-bit immediate + rotate encoding). You'll very often see the pattern:

```
MOVW r0, #0x1234      ; load low 16 bits
MOVT r0, #0x0001      ; load high 16 bits into upper half, low half untouched
```

This is the ARM equivalent of x86 `mov eax, 0x00011234` – two instructions doing one logical load of a 32-bit constant/address. Recognize this pattern instantly; Ghidra usually collapses it into one shown constant but you'll see both instructions in raw listing.

- `LDR Rd, =constant` (pseudo-instruction, assembler resolves to a **literal pool load**, i.e., `LDR Rd, [pc, #offset]` reading a constant stored nearby in code memory) – ARM's answer to not having large immediate encodings for arbitrary values before MOVW/MOVT became standard. Ghidra shows these as `LDR r0, [PC, #0x1c]` with the actual resolved constant annotated – always check the annotation, don't manually compute PC offsets by hand unless Ghidra's cross-reference is broken.
- `ADR Rd, label` – PC-relative address load (position-independent), like `lea reg, [rip+offset]`. Very common in relocatable boot code / PIE binaries. `ADRP` (AArch64 only, not relevant to AArch32 firehose but mention because you'll see it if you ever touch an AArch64 stage) computes page address only.

Arithmetic/logic – all register-to-register, optional third operand can be **shifted** in the same instruction (the "barrel shifter", no x86 equivalent as a free operand modifier):

```
ADD r0, r1, r2, LSL #2      ; r0 = r1 + (r2 << 2) – a single instruction doing what x86 needs lea+shl or two instructions for
```

This fused shift-and-op pattern shows up constantly in array indexing (`arr[i]` where element size is a power of 2) – recognize `LSL #2` as "×4" (int-sized index), `LSL #3` as "×8" (pointer/long index), etc.

Standard ops: `ADD`, `SUB`, `RSB` (reverse subtract, `Rd = Op2 - Rn`, no x86 equivalent, exists because subtracting FROM an immediate isn't otherwise expressible), `AND`, `ORR`, `EOR` (xor), `BIC` (and-not / bit clear, no direct x86 equivalent, `Rd = Rn AND NOT Op2`), `MVN` (move-not, like `not` combined with `mov`), `MUL`, `MLA` (multiply-accumulate, `Rd = Ra + (Rm*Rs)`), `UDIV`/`SDIV` (hardware divide – only on ARMv7-A+ with divide extension; if absent you'll see calls out to a compiler-inserted `__aeabi_idiv` style helper function instead – recognize this pattern as

"this is just a division" even though it looks like a real function call).

Comparison:

- `CMP Rn, Op2` – like `cmp`, sets flags, no result stored. `CMN` – compare negative (`Rn + Op2`, useful for comparing against negative immediates). `TST` – like `test` (AND, flags only). `TEQ` – XOR, flags only, used to check equality while ignoring the N flag from a plain subtract.

Branches:

- `B label` – unconditional jump, like `jmp`.
- `B<cond> label` – conditional jump, e.g. `BEQ`, `BNE`, `BGT`, `BLT`, `BLE`, `BGE`, `BCS` / `BHS`, `BCC` / `BLO`, `BMI`, `BPL` – direct mapping to x86's `je/jne/jg/jl/...`, same semantics, just different mnemonics.
Memorize: `EQ=Z=1`, `NE=Z=0`, `CS/HS=C=1` (unsigned $\geq$), `CC/LO=C=0` (unsigned $<$), `MI=N=1` (negative), `PL=N=0`, `VS/VC=overflow set/clear`, `GT/LT/GE/LE` = signed comparisons (combine N,Z,V).
- `BL label` – call, like `call`, sets LR = return address.
- `BX Rm` – branch and exchange, jumps to address in Rm, **switches ARM/Thumb state based on bit 0 of Rm**. `BX LR` is your `ret`.
- `BLX Rm` / `BLX label` – call with state switch – this is how Thumb code calls into ARM-mode functions and vice versa. If you see a `BLX` where you expected a plain `BL`, that's your signal the target function is compiled in the *other* instruction set from the caller.
- `CBZ Rn, label` / `CBNZ Rn, label` – compare-and-branch-if-(non)zero, Thumb-only, compact single-instruction idiom for `if (r == 0) goto ...` / `if (r != 0) goto ...` with no separate CMP needed. Very common in size-optimized boot code – recognize it immediately, don't go hunting for a preceding CMP that doesn't exist.
- `IT` / `ITT` / `ITE` / etc. (Thumb-2 only) – "if-then" block, conditionalizes the next 1-4 instructions without a branch. E.g. `ITE EQ` followed by two instructions means "if EQ do instr1, else do instr2" – reads like a branchless ternary. Ghidra typically shows the conditional suffix directly on the affected instructions (`MOVEQ`, `MOVNE`) rather than making you track the IT block manually, but knowing the underlying mechanism matters for correctly reading raw bytes or when Ghidra's decompiler gets confused by one.

Load/store – remember, ONLY these instructions touch memory:

- `LDR` / `STR` – 32-bit word. `LDRB` / `STRB` – byte (zero-extended on load). `LDRH` / `STRH` – halfword. `LDRSB` / `LDRSH` – sign-extended byte/halfword loads (x86 `movsx` equivalent; plain `LDRB` / `LDRH` are the `movzx` equivalent).
- Addressing modes – this is where ARM gives you MORE than x86's `[base+index*scale+disp]`:
 - Offset: `LDR r0, [r1, #4]` – like `mov eax, [ecx+4]`.
 - **Pre-indexed**: `LDR r0, [r1, #4]!` – base register r1 is updated (r1 += 4) BEFORE the load. No direct x86 single-instruction equivalent; think of it as `add ecx,4; mov eax,[ecx]` fused.
 - **Post-indexed**: `LDR r0, [r1], #4` – load from [r1] first, THEN r1 += 4. This is the pattern you'll see in loop bodies walking an array/buffer – `mov eax,[ecx]; add ecx,4` fused into one instruction. Recognize post-indexed loads/stores as strong evidence of a pointer-walking loop (e.g., memcpy-style code, which is exactly what you'll be staring at in a physical memory acquisition/dump routine).
 - Register offset with shift: `LDR r0, [r1, r2, LSL #2]` – array indexing, `mov eax, [ecx+edx*4]` equivalent, single instruction.
- `LDM` / `STM` (load/store multiple) – bulk register transfer to/from consecutive memory addresses in one instruction, no x86 equivalent (closest conceptually is a sequence of `pop` / `push`, which is in fact what `PUSH` / `POP` are – they're assembler aliases for `STMDB sp!, {...}` / `LDMIA sp!, {...}`). When you see `LDM r0!, {r1-r4}` in Ghidra, read it as "load 4 words from wherever r0 points, into r1..r4, advance r0 by 16" – this is a fast block-copy idiom and you will see exactly this pattern in memory-dumping/DMA-adjacent firehose code.

5. QEMU + toolchain setup on Arch Linux – actual working setup

This is the confirmed-working setup. The AUR route (`yay -S arm-linux-gnueabi-hf-gcc`) is unreliable, it died on an AUR clone failure. Use the prebuilt ARM toolchain instead, it's the one actually in use.

```
# QEMU user-mode + system-mode ARM emulation, plus binfmt support
sudo pacman -Syu
```

```
sudo pacman -S qemu-user qemu-user-static-binfmt qemu-system-arm
# if pacman throws a file conflict on update (seen with a stale ghidra-
desktop package):
# sudo pacman -Syu --overwrite "*"

# GDB — already ships with arm support on Arch's package
sudo pacman -S gdb

# Cross compiler toolchain — prebuilt binaries direct from ARM, NOT the AUR
package
cd ~/Downloads
wget
https://developer.arm.com/-/media/Files/downloads/gnu/14.2.rel1/binrel/arm-
gnu-toolchain-14.2.rel1-x86_64-arm-none-linux-gnueabihf.tar.xz
tar xf arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-linux-gnueabihf.tar.xz
sudo mv arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-linux-gnueabihf
/opt/arm-toolchain
echo 'export PATH=/opt/arm-toolchain/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

The binary prefix is **arm-none-linux-gnueabihf-**, NOT **arm-linux-gnueabihf-**. The latter was the AUR package name that never actually installed. Every command in this guide uses the correct working prefix — if a command ever errors **command not found** on a ***-gcc** / ***-objdump** / ***-readelf** call, check for that typo from memory first.

Verify:

```
arm-none-linux-gnueabihf-gcc --version
qemu-arm --version
gdb --version    # confirm it reports arm support: `gdb --configuration |
grep arm`
```

Enable transparent execution (so you can literally run an ARM ELF like a native binary and QEMU handles it via binfmt_misc):

```
sudo systemctl restart systemd-binfmt
# test:
file /usr/bin/qemu-arm-static 2>/dev/null || echo "using dynamic qemu-user,
that's fine"
```

6. Exercises – do these, don't just read them

Do them in order. Each one should take you from "I compiled something" to "I can point at a specific instruction in Ghidra and say exactly what it's doing and why the compiler chose it." Don't move to the next exercise until you can do that for the current one without me telling you the answer.

Exercise 1 – Hello world, prologue/epilogue basics

```
// hello.c
#include <stdio.h>
int main(void) {
    printf("hello arm\n");
    return 0;
}
```

```
arm-none-linux-gnueabi-gcc -O0 -marm -static hello.c -o hello_arm
arm-none-linux-gnueabi-gcc -O0 -mthumb -static hello.c -o hello_thumb
qemu-arm ./hello_arm      # should print hello arm
qemu-arm ./hello_thumb
arm-none-linux-gnueabi-objdump -d hello_arm > hello_arm.dis
arm-none-linux-gnueabi-objdump -d hello_thumb > hello_thumb.dis
```

Task: open both `.dis` files. Find `main`. Identify the prologue (`PUSH`/`STMDB`) and epilogue (`POP`/`LDMIA` or separate `BX LR`). Diff the ARM-mode vs Thumb-mode disassembly of the *same source* side by side. Write down (actual notes, not mentally) 3 concrete encoding/instruction differences you observe between the two.

Exercise 2 – Argument passing, prove AAPCS to yourself

```
// args.c
int combine(int a, int b, int c, int d, int e, int f) {
```

```

    return a + b*2 + c*3 + d*4 + e*5 + f*6;
}
int main(void) { return combine(1,2,3,4,5,6); }

```

Compile `-O0 -marm`, disassemble. **Task:** identify which arguments arrive in `r0-r3` and which two arrive via the stack (5th, 6th args). Find the exact `LDR r?, [sp, #N]` instructions pulling the stack args and note the offsets. This proves the "4 in registers, rest on stack" rule to you directly instead of trusting the doc.

Exercise 3 – Callee-saved registers across a nested call

```

// nested.c
int helper(int x) { return x * 3; }
int caller(int a, int b) {
    int t = helper(a);
    return t + b;
}
int main(void) { return caller(4,5); }

```

Task: in `caller`'s disassembly, find where `b` (arriving in `r1`) gets moved into a callee-saved register (likely `r4`) BEFORE the `BL helper` – because `r1` is caller-saved and would get clobbered by the call otherwise. Find the `PUSH {r4, lr}` / `POP {r4, pc}` bracketing this. Explain in your own words why the compiler had to do this move at all – what would break if it didn't.

Exercise 4 – 64-bit values and register pairs

```

// wide.c
long long add64(int a, long long b, int c) { return a + b + c; }
int main(void) { return (int)add64(1, 1000000000000LL, 2); }

```

Task: find where `b` (a `long long`, 64-bit) lands. Confirm it skips a register to stay pair-aligned per the AAPCS rule from section 3. Note exactly which registers/stack slots `a`, `b`, and `c` end up in and explain why there's a "gap."

Exercise 5 – Pointer walking / memcpy-style code (directly relevant to your FYP)

```
// copy.c
void my_copy(unsigned int *dst, unsigned int *src, int n) {
    for (int i = 0; i < n; i++) *dst++ = *src++;
}
int main(void) {
    unsigned int a[4] = {1,2,3,4}, b[4] = {0};
    my_copy(b, a, 4);
    return b[3];
}
```

Compile at both `-O0` and `-O2`. **Task:** at `-O0` you should see explicit `LDR/STR` with separate increments. At `-O2`, look for post-indexed addressing (`LDR r?, [r1], #4` / `STR r?, [r0], #4`) and possibly `LDM/STM` block transfers if the compiler unrolls it. This is the exact instruction pattern you'll be staring down in a firehose read/dump routine – get comfortable recognizing it cold, without Ghidra's decompiler holding your hand.

Exercise 6 – Conditional execution / IT blocks (Thumb-2 specific)

```
// cond.c
int abs_diff(int a, int b) {
    if (a > b) return a - b;
    else return b - a;
}
```

Compile `-mthumb -O2`. **Task:** find the `IT/ITE` block (or the branchless conditional-move sequence the compiler may generate instead at `-O2` – either is a valid finding, note which one you got and why `-O2` might prefer one over the other). Map each conditionally-executed instruction back to the C branch it replaces.

Exercise 7 – Live debugging under QEMU + gdb-multiarch

```
qemu-arm -g 1234 ./hello_arm &
gdb-multiarch ./hello_arm
(gdb) target remote localhost:1234
(gdb) break main
(gdb) continue
(gdb) info registers
(gdb) stepi
(gdb) x/5i $pc
```

Task: single-step through `main` instruction by instruction, watching `r0-r3` and `sp` change in `info registers` after each `stepi`. Correlate what you see live against what you predicted from reading the static disassembly in exercises 1-2. This is the skill you'll actually use when a firehose binary does something Ghidra's decompiler mangles – falling back to raw instruction stepping.

Exercise 8 – Load into Ghidra and cross-check

Import `hello_arm`, `args`, `nested`, `wide`, `copy` (all variants) into a Ghidra project. Auto-analyze. For each, compare Ghidra's decompiler output against your own manual reading from the earlier exercises.

Task: find at least one case where Ghidra's decompiler guesses a variable name/type differently than the real source, and explain from the raw disassembly why Ghidra made that guess (undertyped register, missing prologue analysis, whatever it is). This is the actual skill – not reading clean decompiler output, but knowing when to distrust it and drop to raw instructions, which is exactly the situation you'll be in with a stripped, no-symbols firehose binary.

7. When you actually open a firehose binary

Checklist, in order:

1. Confirm architecture/endianness in Ghidra's import dialog – AArch32, little-endian, is the near-certain default for Snapdragon boot-stage code, but verify rather than assume.
2. Let auto-analysis run fully, then manually check a handful of functions for Thumb vs ARM misdetection (function starts that disassemble to garbage are the tell – right-click, disassemble as Thumb, re-check).
3. Find the entry point / vector table first – boot code always has one, and it'll tell you the reset handler, which is your actual starting point for tracing, not necessarily whatever Ghidra calls `entry`.
4. Look for `SVC` calls (secure monitor entry) and any `BLX` mode switches early – these are your landmarks for "boundary between firehose command handling and lower-level hardware/DMA/memory access," which is precisely the part of the binary you care about for a memory acquisition tool.
5. Expect heavy use of post-indexed loads/stores and `LDM` / `STM` around anything that looks like buffer/memory handling (Exercise 5 pattern) – that's your memory read/write primitive to locate and understand first.

Don't skip to step 5 without doing the exercises. If you can't already predict what a `PUSH {r4,r5,r7,lr}` / post-indexed `LDR` pair means on sight, you'll burn hours in Ghidra second-guessing correct decompiler output instead of trusting your own reading – and firehose binaries won't always come with clean decompilation to lean on.